

The Complete fid-analytical- dashboard ❖❖❖ Line-by-Line Technical Book

The Complete fid-analytical- dashboard — Line-by-Line Technical Book

Purpose. This is the full technical walkthrough of the final production dashboard: every slot, every chart, every helper, every CSS block, every Polymer and JavaScript concept, worked examples with intermediate results at every step, and every important discovery made during debugging. It assumes zero prior knowledge and builds from first principles. It is written so you can explain — and defend — every line to your lead engineer.

The file this documents: fid-analytical-dashboard-FALLBACK.html (the final production version) + fid-dashboard-styles.html (shared CSS).

PART 0 — The foundations everything else stands on

0.1 What a Polymer component is

The whole dashboard is ONE custom HTML element:

```
<dom-module id="fid-analytical-dashboard">
  <template>    ← everything visible (HTML + bindings)
  <script>      ← Polymer({ is: 'fid-analytical-dashboard',
properties, methods })
```

Once registered, `<fid-analytical-dashboard>` can be placed on any Nuxeo page like a built-in tag. Inside it are three kinds of things:

- **properties** — the component's variables (`totalCount`, `startDate`, ...)
- **methods** — its functions (`_formatCount`, `_emailReport`, ...)
- **template** — the markup, wired to properties via *bindings*

0.2 The five Polymer concepts used everywhere

1. One-way binding `[[x]]` — "display this value here." Data flows parent → child only. Example: `[[_formatCount(totalCount)]]`.

2. Two-way binding `{{x}}` — the child element can WRITE the value back into the property. Used wherever data flows *upward*: query results (`data="{{docsPerMonth}}"`, `results-count="{{totalCount}}"`) and form inputs (`value="{{customStart::change}}"` — the `::change` suffix means "write back when the input fires its change event").

3. Computed binding `[[_fn(a, b)]]` — call a method inside the template and use its return value. Polymer automatically re-calls it whenever `a` or `b` changes. This is why the dashboard has no manual "refresh" code — every number on screen is a computed binding that re-runs by itself.

4. Computed property — same idea declared in JS:

```
totalStorageBytes: { type: Number, computed:
  '_sumBuckets(storagePerMonth)' }
```

"totalStorageBytes is always the result of `_sumBuckets(storagePerMonth)`; recalculate whenever `storagePerMonth` changes." Nobody ever assigns it.

5. Observers — "when these properties change, run this method":

```
observers: [
  '_fetchMonthlyMigrated(startDate, endDate)',
  '_onDocsPerMonthChanged(docsPerMonth)',
  '_computeTopByDocs(byDocType)',
  '_computeTopByStorage(storageByDocType)'
],
```

Observers are for *side effects* (firing fetches, deriving several things at once); computed properties/bindings are for *pure single values*.

0.3 The three JavaScript concepts used everywhere

`var self = this;` — inside any `function(){}` passed as a callback (`.map`, `.forEach`, `.then`), `this` no longer refers to the component. Capturing it as `self` first is the ES5-era fix used throughout this file.

Promises and `fetch()` — `fetch(url)` returns a *Promise*: an object representing a value that will arrive later. `.then(fn)` chains the next step; `.catch(fn)` at the end catches a failure from anywhere earlier in the chain. The monthly chart, per-type counts, and the whole email feature run on this.

Guard clauses — nearly every helper starts with `if (!data || !data.length) return ...`. Polymer calls binding helpers *eagerly*, including before any query has returned, so every helper must survive undefined input. This single pattern prevents a whole class of "blank page on load" crashes.

0.4 The mandatory Polymer rule your lead may quiz you on

```
_months: { type: Array, value: function() { return []; } },
```

Array/Object property defaults MUST be wrapped in a function. `value: []` directly would make every instance of the component share ONE array in memory — a classic cross-instance data-corruption bug. The function form gives each instance its own fresh array.

0.5 The `$.=` attribute-binding rule

```
<span class$="[_tabClass(activeView, 'overall')]">
<div style$="[_barStyle(row.value, topByDocsMax)]">
```

`class="..."` sets a JS *property*; `class$="..."` sets the actual HTML *attribute*. For native HTML attributes (`class`, `style`, `href`) you MUST use `$.=` — without it the CSS class silently never applies. The

dashboard uses this for the Growth Overview tabs and the top-5 progress bars.

PART 1 — THE ARCHITECTURE: where every number comes from

1.1 The complete data layer

The dashboard fetches data through exactly THREE mechanisms — and knowing which is which is the key to the whole design:

Mechanism A — Elasticsearch aggregations (7 invisible elements)

`<nuxeo-repository-data>` is a Nuxeo element that draws nothing; it asks Elasticsearch an *aggregation* question and drops the answer into a property via `{{ }}`:

Element (data property)	Question it asks ES
<code>docsPerMonth</code>	documents created, bucketed per month
<code>storagePerMonth</code>	SUM of file bytes, bucketed per month
<code>baselineDocsPerMonth</code>	same as <code>docsPerMonth</code> , baseline range
<code>baselineStoragePerMonth</code>	same as <code>storagePerMonth</code> , baseline range
<code>byDocType</code>	documents grouped by <code>ecm:primaryType</code> (top 10)
<code>storageByDocType</code>	byte SUM grouped by <code>ecm:primaryType</code> (top 10)
<code>byFormat</code>	documents grouped by mime-type (top 10)

Two aggregation styles appear:

- **`with-date-intervals="month" + date-field="dc:created"`** — a *date histogram*: one bucket per month.
- **`grouped-by="ecm:primaryType" + group-limit="10"`** — a *terms aggregation*: one bucket per distinct value, top 10 by count.
- **`metrics="sum(file:content.length)"`** — changes the bucket's number from a document COUNT to a byte SUM.

Every answer has the same shape — an array of buckets:

```
[ { key: '2026-07', value: 8000 }, { key: '2026-08', value: 9000 }, ... ]
```

(For date histograms, key can also arrive as epoch-milliseconds like "1772323200000" — the helpers normalise both forms; see `_monthKey`.)

Critical scale fact: aggregations are computed INSIDE Elasticsearch and return only the buckets — at most 12 (months) or 10 (groups) tiny `{key, value}` pairs — no matter whether the repository holds twenty thousand or a billion documents. Cost scales with bucket count, not corpus size. **Aggregations can ONLY run on Elasticsearch** — a plain database has no equivalent of "bucket by month and sum a field" — so these seven feeds are ES by nature, no configuration needed.

Mechanism B — named Studio page providers (2 count elements)

```
<nuxeo-page-provider auto page-size="1"
```

```

    provider="fid_TOTAL"
    named-query-params="[[_totalParams(startDate, endDate)]]"
    current-page="{{_ignoredTotalPage}}"
    results-count="{{totalCount}}">
</nuxeo-page-provider>

<nuxeo-page-provider auto page-size="1"
  provider="fid_MIGRATED"
  named-query-params="[[_migratedParams(startDate, endDate)]]"
  current-page="{{_ignoredMigratedPage}}"
  results-count="{{migratedCount}}">
</nuxeo-page-provider>

```

Line by line:

- **auto** — fire automatically whenever the bound inputs change; no manual "go" call.
- **page-size="1"** — fetch ONE document but still receive the full, accurate match count. Explained in depth in Part 8.1 (this is the `pageSize=0` vs `1` story).
- **provider="fid_TOTAL"** — names a **Studio-defined page provider** instead of passing raw NXQL. The provider's definition lives in the deployed Studio project, where the **"Use Elasticsearch index" checkbox is ticked** — so its ES routing is guaranteed by its own definition, immune to anything in `nuxeo.conf`.
- **named-query-params="[...]"** — supplies the provider's *named parameters* as a JS object. The helper:

```

  _totalParams: function(startDate, endDate) {
    if (!startDate || !endDate) { return undefined; }
    return {
      dublincore_created: this._toTimestamp(startDate, false),
      dublincore_created_1: this._toTimestamp(endDate, true)
    };
  },

```

The keys `dublincore_created` / `dublincore_created_1` are the parameter names **Studio generated** for the two `dc:created` predicates (`>= start`, `<= end`). Returning `undefined` while either date is missing stops the provider firing on a half-formed range — the modern equivalent of the old "empty query doesn't fire" guard.

- **_toTimestamp(date, isEnd):**

```

  _toTimestamp: function(date, isEnd) {
    if (!date) return date;
    var day = String(date).split('T')[0];
    return day + (isEnd ? ' 23:59:59' : ' 00:00:00');
  },

```

Takes `'2026-07-01'` (or a full ISO datetime — `split('T')[0]` strips any time part defensively) and appends `00:00:00` for a start or `23:59:59` for an end — producing exactly the `'YYYY-MM-DD HH:MM:SS'` format proven working in Bruno.

- **results-count="{{totalCount}}"** — the two-way binding that delivers the count into the component. This one line is the entire output of the element.
- **current-page="{{_ignoredTotalPage}}"** — the element requires a target for the fetched page of documents; we never read it (we only want the count), so it lands in a throwaway property.

Mechanism C — direct REST fetch() calls (monthly + per-type)

The per-month migrated counts and the per-type migrated counts do NOT use any Polymer element at all. They are plain JavaScript `fetch()` calls to the page-provider REST endpoint — the **exact URLs proven in Bruno**:

```
/nuxeo/api/v1/search/pp/fid_MIGRATED/execute
?dublincore_created=2026-07-01 00:00:00
&dublincore_created_1=2026-07-31 23:59:59
&pageSize=1
```

```
/nuxeo/api/v1/search/pp/fid_MIGRATED_BY_TYPE/execute
?dublincore_created=...&dublincore_created_1=...
&system_primaryType=File&pageSize=1
```

Why fetch and not more <nuxeo-page-provider> elements? This is the single most important debugging discovery in the whole project — the "32k bug" — and Part 8.4 tells the full story. The one-line version: repeating provider elements inside a dom-repeat silently failed to deliver their per-month parameters; with named-parameter providers, **missing parameters don't error — Nuxeo silently DROPS those predicates** — so every month's query returned the all-time migrated count (32,000) instead of its own month's count. Building the URL as an explicit string makes dropped parameters *impossible*, and every request is visible in the browser's Network tab.

1.2 The three Studio providers (the server-side half)

All three were built in Nuxeo Studio with the guided predicate editor, all three with **"Use Elasticsearch index" checked**:

Provider	Fixed part	
fid_TOTAL	ecm:isVersion = 0 AND ecm:isTrashed = 0	c c
fid_MIGRATED	same + fid_migration:migrationHistory/*/systemName IS NOT NULL	s
fid_MIGRATED_BY_TYPE	same as fid_MIGRATED	s s

Decoding the fixed part:

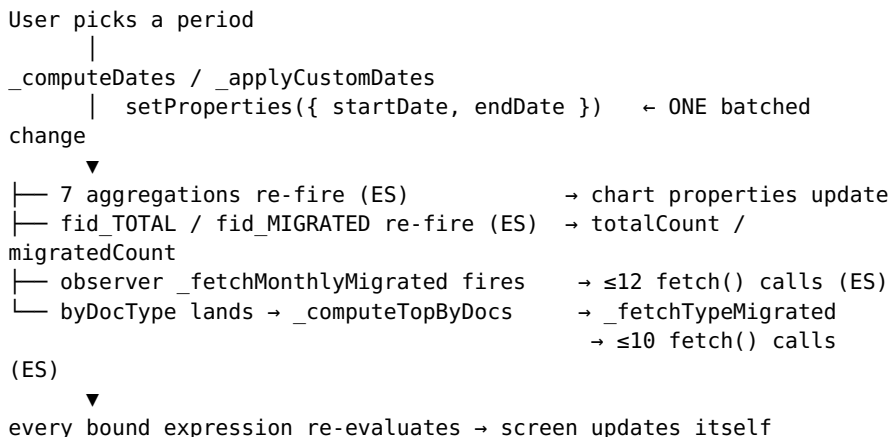
- ecm:isVersion = 0 — ignore archived versions; count only current docs.
- ecm:isTrashed = 0 — ignore trashed docs.
- fid_migration:migrationHistory/*/systemName IS NOT NULL — the business definition of "migrated": the custom fid_migration schema's migrationHistory list has at least one entry with a systemName. The /*/ is NXQL's wildcard into a list of complex properties — "any entry".
- ecm: = **Enterprise Content Management** — Nuxeo's built-in system schema every document carries (ecm:primaryType = the document's type). Because it's a *system* schema, Studio named its parameter system_primaryType (not ecm_primaryType) — discovered during setup and wired into the code accordingly.

Lessons learned creating them (real errors hit, in order):

1. "invalid token SELECT at offset 30" — Studio's Pattern field silently prefixes SELECT * FROM Document WHERE itself; pasting a full SELECT produced a doubled statement. Fix: enter only the predicate body.
2. "lexical error character ? at offset 103" — positional ? placeholders are for hand-written XML providers; **Studio-built providers use NAMED parameters**. Fix: use the guided predicate editor and let Studio generate parameter names.
3. "invalid token IS at offset 233" — structural concatenation issue with the migration predicate in the fixed part, resolved through the

predicate editor. The takeaway your lead will appreciate: **always test a new provider in Bruno with known numbers before wiring it into UI code** — every one of these was caught in Bruno, never in the dashboard.

1.3 The reactive flow — nothing is ever "refreshed" manually



setProperties matters. Setting startDate then endDate as two statements fires every date-bound element TWICE (once per property) — doubling all Elasticsearch traffic on every period change. setProperties batches both into one notification cycle: one fire, always with a self-consistent pair. This was a real found-and-fixed performance bug, and the same pattern is used in ready(), _computeDates, _applyCustomDates, and _applyBaseline.

Worst-case query budget per full page load: 31 — 7 aggregations + 2 provider counts + ≤12 monthly fetches + ≤10 per-type fetches. Every single one is either bucket-bounded or count-only (pageSize=1): **zero document bodies are ever fetched anywhere in this dashboard.**

PART 2 — THE HEADER CARD (period selector + email button)

2.1 Markup, line by line

```
<nuxeo-card class="dates">  
  <div class="header-slot">  
    <div class="header-left">  
      <h1>Analytical Dashboard</h1>  
      <p>Nuxeo usage tracking dashboard</p>  
    </div>  
    <div class="header-divider"></div>  
    <div class="header-controls">
```

<nuxeo-card> is Nuxeo's styled panel element — every slot on the page is one. The header is a flex row: title block, a vertical divider, controls.

```
      <nuxeo-select  
        label="Period"  
        selected="{{selectedPeriod}}"
```

options=['1month', '3months', '6months', '9months', '1year', 'custom']'
on-selected-changed="_onPeriodChanged">

```
</nuxeo-select>
```

- `selected="{{selectedPeriod}}"` — two-way: picking an option writes it into the property.
- `options='[...]'` — note SINGLE quotes around the attribute so the JSON array's double quotes survive. Six choices; five presets + custom.
- `on-selected-changed="_onPeriodChanged"` — Polymer event binding; fires the handler on every selection change.

```
    <div class="period-info-wrap">
      <iron-icon class="period-info-icon" icon="icons:info-
outline"></iron-icon>
      <div class="period-tooltip"> The period is calculated from
today's date ... </div>
    </div>
```

A pure-CSS tooltip: the icon plus a hidden div that CSS reveals on hover (`.period-info-wrap:hover .period-tooltip { opacity: 1; }` in the styles file). No JavaScript involved.

```
    <template is="dom-if" if="[[_isCustom(selectedPeriod)]]">
      <div class="custom-dates">
        <div class="date-field"><label>Start Date</label>
        <input type="date" value="{{customStart::change}}" /></div>
        <div class="date-field"><label>End Date</label>
        <input type="date" value="{{customEnd::change}}" /></div>
        <div class="date-field"><label>&nbsp;</label>
        <button type="button" class="custom-submit" on-
tap="_applyCustomDates">Apply</button></div>
      </div>
    </template>
```

- **dom-if** — the custom pickers exist in the DOM ONLY while `selectedPeriod === 'custom'`. Not hidden — genuinely absent otherwise.
- `_isCustom` is a one-liner: `return period === 'custom';`
- `value="{{customStart::change}}"` — event-qualified two-way binding: write the input's value back when its change event fires (i.e. when the user commits a date, not on every keystroke).
- `<label> </label>` — an empty label above the Apply button so all three columns align vertically. A tiny layout trick worth knowing.
- The **Apply button** is the scale-safety design: dates only reach `startDate/endDate` (and therefore fire queries) when clicked — typing in the pickers costs zero queries.

2.2 The date pipeline, line by line

```
selectedPeriod: { type: String, value: '1month', observer:
'_onPeriodChanged' },
```

A per-property observer (an alternative to the `observers`: array for a single property): whenever `selectedPeriod` changes, call `_onPeriodChanged(newValue)`.

```
_onPeriodChanged: function(newVal) {
  if (!newVal || newVal === 'custom') return;
  this._computeDates(newVal);
},
```

Presets recompute dates immediately. 'custom' deliberately does nothing — it waits for the Apply button.

```
_computeDates: function(period) {
  var monthMap = { '1month': 1, '3months': 3, '6months': 6,
'9months': 9, '1year': 12 };
  var months = monthMap[period];
```

```

    if (!months) return;
    var end = new Date();
    var start = new Date();
    start.setMonth(start.getMonth() - months);
    this.setProperties({
      startDate: this._toISODate(start),
      endDate: this._toISODate(end)
    });
  },

```

- monthMap — a lookup object mapping the option string to a number of months; cleaner than an if/else ladder.
- start.setMonth(start.getMonth() - months) — JavaScript's Date rolls over correctly (subtracting 3 months from January lands in the previous October).
- **The batched setProperties** — the anti-double-fire fix (Part 1.3).

```

    _applyCustomDates: function() {
      if (this.customStart && this.customEnd) {
        this.setProperties({ startDate: this.customStart, endDate:
this.customEnd });
      }
    },

```

Custom commits both ends together, same batching, only when both are filled.

```

    _toISODate: function(date) { return date.toISOString().split('T')
[0]; },

```

Date → 'YYYY-MM-DD' by cutting the ISO string at the T.

```

    _extendEndDate: function(date) {
      if (!date) return date;
      var d = new Date(date);
      d.setDate(d.getDate() + 1);
      d.setMilliseconds(d.getMilliseconds() - 1);
      return d.toISOString().split('T')[0];
    },

```

Used by every aggregation's end-date. **Why it exists:** a date string like '2026-07-31' means *midnight at the start* of the 31st — which would exclude the entire last day's documents. This helper nudges to "one millisecond before the NEXT day," making the end date inclusive. Every aggregation binds end-date="[_extendEndDate(endDate)]" for this reason.

2.3 Header CSS (the essentials)

```

.header-slot { display: flex; align-items: center; gap: 24px; }
.header-divider { width: 1px; align-self: stretch; background:
#e0e0e0; }
.custom-dates { display: flex; align-items: flex-end; gap: 12px; }
.date-field { display: flex; flex-direction: column; gap: 4px; }

```

Flexbox rows; align-items: flex-end bottom-aligns the pickers with the Apply button; the divider stretches to full header height via align-self: stretch.

PART 3 — THE KPI STRIP (the four headline numbers)

3.1 Markup


```

<nuxeo-card class="full-width">
  <div class="kpi-strip">
    <div class="kpi-cell">
      <span class="kpi-value">[[_formatCount(totalCount)]]</span>
      <span class="kpi-label">New documents created</span>
    </div>
    <div class="kpi-divider"></div>
    <div class="kpi-cell">
      <span class="kpi-value">[[_formatStorage(totalStorageBytes)]]
</span>
      <span class="kpi-label">of storage added</span>
    </div>
    <div class="kpi-divider"></div>
    <div class="kpi-cell">
      <span class="kpi-value">[[_formatCount(migratedCount)]]</span>
      <span class="kpi-label">From migrated use cases</span>
    </div>
    <div class="kpi-divider"></div>
    <div class="kpi-cell">
      <span class="kpi-value">[[_formatNonMigrated(totalCount,
migratedCount)]]</span>
      <span class="kpi-label">From non-migrated use cases</span>
    </div>
  </div>
</nuxeo-card>

```

Four cells, three dividers, all computed bindings. Where each number comes from:

KPI	Source
New documents	<code>totalCount</code> ← <code>fid_TOTAL</code> provider (ES)
Storage added	<code>totalStorageBytes</code> ← computed from <code>storagePerMonth</code>
Migrated	<code>migratedCount</code> ← <code>fid_MIGRATED</code> provider (ES)
Non-migrated	derived: <code>total</code> – <code>migrated</code>

3.2 The helpers, line by line

```

totalStorageBytes: { type: Number, computed:
'_sumBuckets(storagePerMonth)' },

_sumBuckets: function(data) {
  if (!data || !data.length) return 0;
  return data.reduce(function(s, e) { return s + (e.value || 0); },
0);
},

```

A deliberate performance decision: total storage is NOT a separate query. The per-month storage histogram is already fetched for the Growth chart — summing its buckets client-side gives the period total for free. `.reduce()` walks the array carrying an accumulator `s` (starting 0) and adds every bucket's value. One query eliminated.

```

_formatCount: function(n) {
  n = n || 0;
  if (n >= 1000000000) return (Math.round(n / 1000000000) / 10) +
'B';
  if (n >= 1000000) return (Math.round(n / 1000000) / 10) + 'M';
  if (n >= 1000) return (Math.round(n / 100) / 10) + 'K';
  return String(n);
},

```

Compact human formatting with one decimal. The math trick: to show 15416 as 15.4K, divide by 100 → 154.16, round → 154, divide by 10 → 15.4. Dividing by 100-then-10 instead of straight 1000 is what

preserves exactly one decimal place. Ladder order matters — largest unit first wins. Ready for billions (1.2B) out of the box.

```
_formatNonMigrated: function(total, migrated) {  
  var n = (total || 0) - (migrated || 0);  
  if (n < 0) n = 0;  
  ...same K/M/B ladder...  
},
```

Subtraction with a floor at zero — if the two counts ever momentarily disagree mid-load (one query answered, the other not yet), the KPI shows 0 rather than a negative number.

_formatStorage(bytes) — the byte ladder (B → KB → MB → GB → TB → PB), each threshold the exact power of 1024, first match wins, one decimal:

```
if (bytes >= 1099511627776) return (Math.round(bytes / 1099511627776  
* 10) / 10) + ' TB';  
if (bytes >= 1073741824) return (Math.round(bytes / 1073741824 *  
10) / 10) + ' GB';  
...
```

3.3 Worked example

Query results land: totalCount = 15416, migratedCount = 5095, storagePerMonth = [{value: 524288000}, {value: 786432000}, {value: 419430400}].

```
_sumBuckets → 524288000+786432000+419430400 = 1730150400 →  
totalStorageBytes  
_formatCount(15416) → 15416/100=154.16 → round 154 → /10 →  
'15.4K'  
_formatStorage(1730150400) → ≥1073741824 →  
1730150400/1073741824=1.611... → '1.6 GB'  
_formatCount(5095) → '5.1K'  
_formatNonMigrated(15416, 5095) → 10321 → '10.3K'
```

Strip shows: **15.4K | 1.6 GB | 5.1K | 10.3K** — and every one recomputes automatically the instant its source property changes.

3.4 CSS

```
.kpi-strip { display: flex; align-items: stretch; }  
.kpi-cell { flex: 1; display: flex; flex-direction: column; align-  
items: center; }  
.kpi-value { font-size: 28px; font-weight: 700; color: #1E5631; }  
.kpi-label { font-size: 12px; color: #7a8580; text-transform:  
uppercase; letter-spacing: .4px; }  
.kpi-divider { width: 1px; background: #e6ebe8; }
```

flex: 1 makes all four cells equal width regardless of content; dividers are 1-pixel stretched columns between them.

PART 4 — SLOT 1: GROWTH OVERVIEW (the hero chart with four tabs)

4.1 The tab system, line by line

```
<nuxeo-card class="full-width" heading="Growth Overview">  
  <div class="view-tabs">
```

```

        <span class$="[[_tabClass(activeView, 'overall')]]" on-
tap="_selectOverall">Overall</span>
        <span class$="[[_tabClass(activeView, 'docType')]]" on-
tap="_selectDocType">By Doc Type</span>
        <span class$="[[_tabClass(activeView, 'format')]]" on-
tap="_selectFormat">By Format</span>
        <span class$="[[_tabClass(activeView, 'size')]]" on-
tap="_selectSize">By Size</span>
    </div>

    activeView: { type: String, value: 'overall' },

    _tabClass: function(active, view) {
        return active === view ? 'view-tab active' : 'view-tab';
    },
    _selectOverall: function() { this.activeView = 'overall'; },
    _selectDocType: function() { this.activeView = 'docType'; },
    _selectFormat: function() { this.activeView = 'format'; },
    _selectSize: function() { this.activeView = 'size'; },

```

The entire tab machinery is one string property. Each `<span>`:

- `on-tap` sets `activeView` to its view name;
- `class$=` (note the `$` — attribute binding, Part 0.5) recomputes for EVERY tab whenever `activeView` changes: the matching tab gets the extra active class (green underline in CSS), the others lose it.

Each tab body is wrapped in a `dom-if`:

```

<template is="dom-if" if="[[_isView(activeView, 'overall')]]"> ...
</template>

_isView: function(active, view) { return active === view; },

```

Only the selected view's chart exists in the DOM — the other three aren't hidden, they're absent, so no off-screen charts consume memory or redraw time. **Important cross-feature detail:** the email/PDF exporter needs the Overall chart's canvas, so `_emailReport` sets `this.activeView = 'overall'` and waits 500 ms before capturing — because a `dom-if`'d chart that isn't active has no canvas to capture (Part 9).

4.2 Tab 1 — OVERALL: storage bars + documents line together

```

<chart-bar
  labels="[[_monthLabels(storagePerMonth)]]"
  values="[[_overallValues(storagePerMonth, docsPerMonth,
totalCount)]]"
  series="[[_overallSeries(storagePerMonth)]]"
  options="[[_overallOptions(storagePerMonth)]]">
</chart-bar>

```

`<chart-bar>` is Nuxeo's Chart.js wrapper: `labels` = x-axis, `values` = an array of data series, `series` = legend names, `options` = Chart.js config.

`_monthLabels` — x-axis

```

_monthLabels: function(data) {
    if (!data || !data.length) return [];
    var self = this;
    return data.map(function(entry) { return
self._toMonthYear(entry.key); });
},

```

Maps every bucket key through `_toMonthYear` (Part 7.4) $\rightarrow$ ['May-26', 'Jun-26', 'Jul-26']. Note the `self = this` pattern — `_toMonthYear` must be called on the component from inside the `.map` callback.

`_overallValues` — the two data series

```
_overallValues: function(storage, docs, total) {
  if (!storage || !storage.length) return [[]];
  var unit = this._storageUnit(storage);
  var storageVals = storage.map(function(entry) {
    var v = (entry.value || 0) / unit.divisor;
    return Math.round(v * 100) / 100;
  });
  var rawDocs = this._docsAlignedToStorage(storage, docs);
  var docVals = this._reconcileDocs(rawDocs, total);
  return [storageVals, docVals];
},
```

Line by line:

- Guard clause (survives the pre-data render).
- `_storageUnit(storage)` — inspects the LARGEST bucket and picks one unit (KB/MB/GB/TB) for the whole chart (Part 7.3). Nothing is hardcoded "MB" — the unit follows the data, which is what makes this chart valid from a 20 KB dev instance to a multi-TB production one.
- Every byte value $\div$ divisor, rounded to 2 decimals ($\times 100$, round, $\div 100$).
- `_docsAlignedToStorage` — the alignment safeguard:

```
_docsAlignedToStorage: function(storage, docs) {
  var docMap = {};
  (docs || []).forEach(function(entry) { docMap[entry.key] =
entry.value; });
  return (storage || []).map(function(entry) { return
docMap[entry.key] || 0; });
},
```

Storage and documents come from two SEPARATE histograms that are not guaranteed to return identical month sets (a month with zero uploads may be absent from one). Zipping them by array position would silently shift later months against each other. Instead: build a lookup keyed by month, then walk the STORAGE months (the reference order) and look up each month's doc count **by key** — missing months become 0.

- `_reconcileDocs(rawDocs, total)` — nudges the histogram's monthly doc counts so their SUM equals the authoritative `totalCount` from `fid_TOTAL`:

```
_reconcileDocs: function(rawMonthly, authoritativeTotal) {
  var aggTotal = rawMonthly.reduce(function(s, v) { return s + v;
}, 0);
  if (!authoritativeTotal || aggTotal === 0 || aggTotal ===
authoritativeTotal) {
    return rawMonthly; // nothing to fix
  }
  var factor = authoritativeTotal / aggTotal; // proportional
scale
  var scaled = rawMonthly.map(function(v) { return Math.round(v *
factor); });
  var scaledTotal = scaled.reduce(function(s, v) { return s + v;
}, 0);
  var drift = authoritativeTotal - scaledTotal; // rounding
residue
  if (drift !== 0 && scaled.length) {
    var maxIdx = 0; // give residue
to biggest month
    for (var i = 1; i < scaled.length; i++) {
```

```

        if (scaled[i] > scaled[maxIdx]) maxIdx = i;
    }
    scaled[maxIdx] += drift;
    if (scaled[maxIdx] < 0) scaled[maxIdx] = 0;
}
return scaled;
},

```

Reading it: if the histogram sum already equals the KPI total, pass through untouched. Otherwise scale every month by the ratio, then, because per-month rounding can leave the sum a few off, add the leftover (drift) to the largest month so the chart's months add up EXACTLY to the KPI number the user sees two cards above.

Honest history note (a debugging discovery your lead may ask about): this same reconcile-scaling technique was originally ALSO used for the Monthly Migration chart's totals — and there it caused the infamous inflation bug (one misaligned bucket → survivors scaled up → a month showing 32k against a 10k total). It was removed from the monthly path (Part 6 uses raw histogram values); it remains here on the Overall chart only, where the doc series is cosmetic alignment against an authoritative total rather than an input to a subtraction.

- Returns [storageVals, docVals] — two series → grouped bars/lines.

overallSeries and overallOptions

```

_overallSeries: function(storage) {
    return ['Storage added in (' + this._storageUnit(storage).name +
    ')',
        'Number of documents created'];
},

```

Legend text, index-matched to the values array. It calls the SAME _storageUnit as _overallValues, so the legend's unit can never disagree with the numbers actually plotted.

_overallOptions(storage) returns the Chart.js configuration object — responsive sizing, legend at top, no animation ("animation": false, a deliberate render-performance choice so every data update paints instantly instead of tweening), light gridlines, index-mode tooltips (hover one month, see both series). Taking storage as an argument makes Polymer re-evaluate it when data changes.

4.3 Tabs 2 & 3 — the two pies (By Doc Type / By Format)

```

<template is="dom-if" if="[[!_isEmpty(byDocType)]]">
  <chart-pie labels="[[_labels(byDocType)]]" values="
[[_values(byDocType)]]" options='{...}'>
  </chart-pie>
</template>
<template is="dom-if" if="[[!_isEmpty(byDocType)]]">
  <div class="chart-empty">No documents found in this period</div>
</template>

```

The empty-state pattern used across the dashboard: a pair of complementary dom-ifs (!_isEmpty / _isEmpty) — chart when there's data, a friendly message when there isn't, never a broken blank chart.

```

_isEmpty: function(data) { return !data || !data.length; },
_labels: function(data) {
    if (!data || !data.length) return [];
    return data.map(function(entry) { return entry.key; });
},
_values: function(data) {
    if (!data || !data.length) return [[]];

```

```

    return [data.map(function(entry) { return entry.value; })];
  },

```

`_labels` = raw bucket keys (type names). `_values` — note the extra []: Chart.js expects an ARRAY of series even for a single-series pie, so the mapped array is wrapped once.

By Format uses `_typeLabels` instead — mime-types are ugly, so:

```

    _typeLabels: function(data) {
      if (!data || !data.length) return [];
      var self = this;
      return data.map(function(entry) { return
self._friendlyType(entry.key); });
    },
    _friendlyType: function(mime) {
      if (!mime) return 'Unknown';
      var map = { 'application/pdf': 'PDF', 'image/png': 'PNG Image',
        'application/vnd.openxmlformats-officedocument.wordprocessingml.document': 'Word',
        ... };
      if (map[mime]) return map[mime];
      var parts = mime.split('/');
      return parts.length > 1 ? parts[1].toUpperCase() : mime;
    },

```

A lookup table for common types, then a graceful fallback: an unknown application/x-foo becomes X-F00 (subtype uppercased) rather than the full unreadable mime string.

4.4 Tab 4 — BY SIZE (storage bars alone)

```

    _storageValues: function(data) {
      if (!data || !data.length) return [[]];
      var unit = this._storageUnit(data);
      return [data.map(function(entry) {
        var v = (entry.value || 0) / unit.divisor;
        return Math.round(v * 100) / 100;
      })];
    },
    _storageSeries: function(data) {
      return ['Storage (' + this._storageUnit(data).name + ')'];
    },

```

Same auto-scaling recipe as Overall's storage series, single-series form.

4.5 Worked example — the Overall tab end to end

Inputs (3-month period):

```

storagePerMonth = [ {key: '2026-05', value: 524288000}, // 500 MB
                    {key: '2026-06', value: 786432000}, // 750 MB
                    {key: '2026-07', value: 419430400} ] // 400 MB

docsPerMonth    = [ {key: '2026-05', value: 5100},
                    {key: '2026-06', value: 6100}, // note:
sums to 15600
                    {key: '2026-07', value: 4400} ]

totalCount      = 15416 //
fid_TOTAL (authoritative)

_monthLabels(storage)      → ['May-26', 'Jun-26', 'Jul-26']
_storageUnit(storage)     → max 786432000 ≥ 1048576 →
{name: 'MB', divisor: 1048576}
storageVals              → [500, 750, 400]

```

```

_docsAlignedToStorage(storage, docs)
  docMap = {'2026-05':5100,'2026-06':6100,'2026-07':4400}
  walk storage months      → [5100, 6100, 4400]
_reconcileDocs([5100,6100,4400], 15416)
  aggTotal = 15600 ≠ 15416 → factor = 15416/15600 = 0.98821
  scaled   = [5040, 6028, 4348] (each × factor, rounded)
  scaledTotal = 15416 → drift 0 → done
returns [[500,750,400], [5040,6028,4348]]
_overallSeries      → ['Storage added in (MB)',
'Number of documents created']

```

Chart shows three months, storage bars in MB, document counts summing to exactly the KPI's 15,416.

4.6 CSS for the tabs

```

.view-tabs { display: flex; gap: 18px; border-bottom: 1px solid
#e6ebe8; }
.view-tab { padding: 8px 2px; font-size: 13px; color: #7a8580;
cursor: pointer;
border-bottom: 2px solid transparent; }
.view-tab.active { color: #1E5631; border-bottom-color: #2e7d32;
font-weight: 600; }
.chart-box { position: relative; height: 320px; }
.chart-empty { padding: 48px 0; text-align: center; color: #9aa39c;
}

```

Inactive tabs carry a TRANSPARENT bottom border of the same width as the active green one — so switching tabs never shifts the layout by 2px. `.chart-box` gives Chart.js a fixed-height positioned container (required for `maintainAspectRatio:false` to work properly).

PART 5 — SLOT 2: MIGRATION ADOPTION (the donut)

5.1 Markup, line by line

```

<nuxeo-card class="card-onethird" heading="Migration Adoption">
  <div class="donut-wrap">
    <chart-pie
      labels="[_migrationLabels()]"
      values="[_migrationValues(totalCount, migratedCount)]"
      colors='["#2e7d32", "#d32f2f"]'
      options='{ "responsive":true, "maintainAspectRatio":false, "cutoutPercentage":68,
        "legend":
{"display":true, "position":"bottom", ...}, "animation":false, ...}'>
    </chart-pie>
    <div class="donut-center">
      <span class="pct">[_migratedPct(totalCount, migratedCount)]%
    </span>
      <span class="pct-label">MIGRATED</span>
    </div>
    <div>
      <div class="migration-summary">
        <div class="stat"><span class="num">
[_ _withCommas(migratedCount)]</span><span
class="lbl">Migrated</span></div>
        <div class="stat"><span class="num">
[_ _nonMigratedCommas(totalCount, migratedCount)]</span><span
class="lbl">Non-Migrated</span></div>

```

```

        <div class="stat"><span class="num">[[_withCommas(totalCount)]]
</span><span class="lbl">Total</span></div>
    </div>
</nuxeo-card>

```

- "cutoutPercentage": 68 — the Chart.js option that turns a pie into a donut: 68% of the radius is hollow.
- colors=['"#2e7d32"', "#d32f2f"'] — fixed semantic colors: green = migrated, red = non-migrated. The SAME two hex codes are reused by the Monthly Migration chart (_monthlyColors) so the two visualisations speak one color language.
- **.donut-center — the overlay trick.** The percentage in the middle is NOT drawn by Chart.js; it's an ordinary HTML <div> absolutely positioned over the canvas hole:

```

        .donut-wrap { position: relative; height: 220px; }
        .donut-center { position: absolute; top: 50%; left: 50%;
                        transform: translate(-50%, -50%); text-align:
center;
                        pointer-events: none; }

```

translate(-50%, -50%) centers the div on its own midpoint; pointer-events: none lets mouse hovers pass through to the chart's tooltips.

This overlay is also the reason the email PDF has a special donut-center step (Part 9): canvas.toDataURL() captures only the canvas, so the exported donut has an empty hole — the PDF code re-draws the percentage as text at the image's computed centre.

5.2 Helpers, line by line

```

    _migrationLabels: function() { return ['Migrated', 'Non-Migrated'];
},

    _migrationValues: function(total, migrated) {
        return [[migrated || 0, this._nonMigrated(total, migrated)]];
    },

    _nonMigrated: function(total, migrated) {
        var diff = (total || 0) - (migrated || 0);
        return diff > 0 ? diff : 0;
    },

    _migratedPct: function(total, migrated) {
        var t = total || 0;
        if (t <= 0) return 0; // division-by-zero
guard
        return Math.round(((migrated || 0) / t) * 100);
    },

    _withCommas: function(n) {
        n = n || 0;
        return String(n).replace(/\B(?=(\d{3})+(?!\d))/g, ',');
    },

    _nonMigratedCommas: function(total, migrated) {
        var diff = (total || 0) - (migrated || 0);
        if (diff < 0) diff = 0;
        return String(diff).replace(/\B(?=(\d{3})+(?!\d))/g, ',');
    },

```

- _migrationValues returns [[a, b]] — one series with two slices.
- _nonMigrated floors at 0 — mid-load, migrated can momentarily exceed total; the donut must never render a negative slice.
- _migratedPct guards t <= 0 (empty period → 0%, not NaN%).
- _withCommas's regex, in words: at every position between digits that is followed by one-or-more complete groups of three digits running to the end of the number, insert a comma. 5095 → 5,095; 1234567 →

1,234,567. You don't memorise the regex — you describe what it does.

5.3 Worked example

totalCount = 15416, migratedCount = 5095:

```
_migrationValues → [[5095, _nonMigrated(15416,5095)=10321]] → slices
5095 green / 10321 red
_migratedPct      → 5095/15416 = 0.3305 → ×100 → round → 33 →
center shows '33%'
_withCommas(5095)      → '5,095'
_nonMigratedCommas(...) → '10,321'
_withCommas(15416)     → '15,416'
```

Both count inputs come from ES-backed Studio providers — this donut is the dashboard's ground truth, and the Monthly Migration chart (next part) is deliberately built so its per-month numbers can never contradict it.

PART 6 — SLOT 3: MONTHLY MIGRATION (the chart that fought back)

This chart went through THREE architectures before landing on the final one. Understanding *why* the first two failed is the best Polymer lesson in this whole project — and very likely what your lead will find most interesting.

6.1 The final markup

```
<nuxeo-card class="card-twothirds" heading="Monthly Migration">
  <div class="chart-sub-title">Migrated vs non-migrated documents
per month</div>
  <template is="dom-if" if="[[!_monthlyEmpty(monthlyMigrated)]]">
    <div class="chart-box">
      <chart-bar
        labels="[[_monthlyLabels(monthlyMigrated)]]"
        values="[[_monthlyValues(monthlyMigrated)]]"
        series="[[_monthlySeries()]]"
        colors="[[_monthlyColors()]]"
        options='{...same axis/legend/tooltip config as the other
bar charts...}'>
      </chart-bar>
    </div>
  </template>
  <template is="dom-if" if="[[_monthlyEmpty(monthlyMigrated)]]">
    <div class="chart-empty">No documents found in this period</div>
  </template>
</nuxeo-card>
```

Notice there are **NO data-fetching elements for this chart in the template at all** — the data arrives via JavaScript `fetch()` (Mechanism C). The chart binds purely to the `monthlyMigrated` rows.

6.2 The display helpers

```
_monthlyLabels: function(rows) {
  if (!rows || !rows.length) return [];
  return rows.map(function(r) { return r.label; });
}
```

```

    },
    _monthlyValues: function(rows) {
      if (!rows || !rows.length) return [];
      var migrated = rows.map(function(r) { return r.migrated || 0;
    });
      var nonMigrated = rows.map(function(r) { return r.nonMigrated ||
0; });
      return [migrated, nonMigrated];
    },
    _monthlySeries: function() { return ['Migrated', 'Non-Migrated']; },
    _monthlyColors: function() { return ['#2e7d32', '#d32f2f']; },
    _monthlyEmpty: function(rows) { return !rows || !rows.length; },

```

Straight projections of the row objects into Chart.js's two series, with the donut's exact green/red pair.

6.3 The data pipeline — properties and observers

```

_months: { type: Array, value: function() { return []; } },
_monthGen: { type: Number, value: 0 },
monthlyMigrated: { type: Array, value: function() { return []; } },

observers: [
  '_fetchMonthlyMigrated(startDate, endDate)',
  '_onDocsPerMonthChanged(docsPerMonth)',
  ...
],

```

- `_months` — one descriptor per month in the period: {key: '2026-07', label: 'Jul-26', start: '2026-07-01 00:00:00', end: '2026-07-31 23:59:59', migrated:0}.
- `_monthGen` — a **request generation counter**, the race guard (6.5).
- `monthlyMigrated` — the finished rows the chart binds to.
- Observer 1: any date change rebuilds the months and fires the fetches.
- Observer 2: whenever the totals histogram (`docsPerMonth`, already fetched for the Growth chart) arrives or changes, re-merge.

6.4 `_buildMonthList` — splitting the period into months, line by line

```

_buildMonthList: function(startDate, endDate) {
  var start = this._parseLocal(startDate);
  var end = this._parseLocal(endDate);
  var months = [];
  var cursor = new Date(start.getFullYear(), start.getMonth(), 1);
  var last = new Date(end.getFullYear(), end.getMonth(), 1);
  while (cursor <= last) {
    var y = cursor.getFullYear();
    var mo = cursor.getMonth();
    var firstOfMonth = new Date(y, mo, 1);
    var lastOfMonth = new Date(y, mo + 1, 0);
    var mStart = firstOfMonth < start ? start : firstOfMonth;
    var mEnd = lastOfMonth > end ? end : lastOfMonth;
    var mk = y + '-' + ('0' + (mo + 1)).slice(-2);
    months.push({
      key: mk,
      label: this._toMonthYear(mk + '-01'),
      start: this._toTimestamp(this._toISOLocal(mStart), false),
      end: this._toTimestamp(this._toISOLocal(mEnd), true),
      migrated: 0
    });
    cursor.setMonth(cursor.getMonth() + 1);
  }
}

```

```

    return months;
  },

```

- `_parseLocal` (helper): splits 'YYYY-MM-DD' and builds new `Date(y, m-1, d)` — a LOCAL-time date. Why not new `Date('2026-07-01')` directly? Because the string form parses as **UTC midnight**, which in timezones behind UTC lands on the *previous local day* — historically this shifted the whole month walk by one. Constructing from parts sidesteps it.
- cursor starts at the 1st of the start month; last is the 1st of the end month; the while walks month by month.
- `new Date(y, mo + 1, 0)` — **the day-zero trick**. Day 0 of NEXT month = the LAST day of THIS month. Handles 28/29/30/31-day months and leap years in one line, no lookup tables.
- Edge clamping: the first month starts at the period's actual start date (not necessarily the 1st), the last month ends at the actual end date — so a custom range "July 10 → Sep 20" queries exactly those bounds.
- `('0' + (mo + 1)).slice(-2)` — zero-padding: month 6 (July, 0-indexed) → '0'+7='07' → key '2026-07'.
- `cursor.setMonth(cursor.getMonth() + 1)` — JS Date rolls the year over automatically (Dec + 1 → next January).

6.5 _fetchMonthlyMigrated — the engine, line by line

```

_fetchMonthlyMigrated: function(startDate, endDate) {
  if (!startDate || !endDate) {
    this._months = [];
    this.monthlyMigrated = [];
    return;
  }
  var months = this._buildMonthList(startDate, endDate);
  this._months = months;
  this._monthGen = (this._monthGen || 0) + 1;
  var gen = this._monthGen;
  this._renderMonthly();
  var self = this;
  months.forEach(function(m) {
    var url = '/nuxeo/api/v1/search/pp/fid_MIGRATED/execute'
      + '?dublincore_created=' + encodeURIComponent(m.start)
      + '&dublincore_created_1=' + encodeURIComponent(m.end)
      + '&pageSize=1';
    fetch(url, {
      headers: { 'Accept': 'application/json' },
      credentials: 'include'
    }).then(function(resp) {
      if (!resp.ok) { throw new Error('fid_MIGRATED HTTP ' +
resp.status); }
      return resp.json();
    }).then(function(json) {
      if (self._monthGen !== gen) { return; }
      var count = json && typeof json.resultsCount === 'number' ?
json.resultsCount : 0;
      m.migrated = count > 0 ? count : 0;
      self._renderMonthly();
    }).catch(function(err) {
      if (window.console) { console.error('Monthly migrated fetch
failed for ' + m.key, err); }
      if (self._monthGen !== gen) { return; }
      self._renderMonthly();
    });
  });
},

```

Walking it:

1. Missing dates → clear everything, done.
2. Build the month list, store it.
3. **gen — the race guard**. Bump the generation counter and capture its value in this closure. If the user changes the date range while these requests are still in flight, a NEW call bumps the counter — and every response handler's first line, `if (self._monthGen !== gen) return;`, makes stale responses drop themselves silently. Without this, a slow July response from the OLD range could overwrite the NEW range's chart.
4. `_renderMonthly()` immediately — the chart appears at once with `migrated = 0` per month, then fills in as responses land (progressive rendering, no blank wait).
5. Per month: build the URL **as an explicit string** — `encodeURIComponent` safely encodes the space in `'2026-07-01 00:00:00'` as `%20`. Because the parameters are literal text in the URL, they **cannot be silently dropped** — the fix for the 32k bug (Part 8.4).
6. credentials: `'include'` — send the Nuxeo session cookie, so the call is authenticated as the logged-in user (same as every other dashboard request).
7. Response handling: `non-2xx` → `throw` → the `.catch`. Success → validate `resultsCount` is a number, **clamp negatives to 0** (a page provider is allowed to report a negative count meaning "unknown"; the chart must never plot -1), write it onto the month object, re-render.
8. `.catch` — log the real error with the month key for diagnosis, and still re-render so one failed month shows 0 instead of freezing the whole chart.

6.6 `_renderMonthly` — the merge, line by line

```

    _onDocsPerMonthChanged: function() { this._renderMonthly(); },

    _renderMonthly: function() {
      var months = this._months || [];
      if (!months.length) { this.monthlyMigrated = []; return; }
      var self = this;
      var totalMap = {};
      (this.docsPerMonth || []).forEach(function(b) {
        totalMap[self._monthKey(b.key)] = b.value || 0;
      });
      this.monthlyMigrated = months.map(function(m) {
        var migrated = m.migrated || 0;
        var all = totalMap[m.key];
        if (typeof all !== 'number') { all = migrated; } // total not
in yet
        if (migrated > all) { all = migrated; }           // never
negative
      });
      return {
        key:      m.key,
        label:    m.label,
        migrated: migrated,
        nonMigrated: all - migrated,
        all:      all
      };
    });
  },

```

- Builds a totals lookup from `docsPerMonth`, normalising every bucket key through `_monthKey` (6.7) so `'2026-07'`, `'2026-07-01'`, and epoch-ms keys all land on the same `'YYYY-MM'`.
- Per month: `nonMigrated = total - migrated`, with two guards:
 - **total not arrived yet** → treat total as = migrated (non-migrated 0) rather than crashing or showing negative;
 - **migrated > total** (possible transiently from indexing lag) → raise total to migrated, again pinning non-migrated at 0.
- Assigns a **NEW array** to `monthlyMigrated` — a fresh reference,

which is what reliably triggers Polymer to re-evaluate every chart binding. (This is the lesson from failed architecture #2: mutating elements *inside* an existing array does not reliably notify.)

- **Why this design cannot reproduce the inflation bug:** each month's non-migrated is computed from *that month's own* total minus *that month's own* migrated — there is no cross-month scaling step, so no misalignment can ever inflate a neighbour. And summed across months, the values agree with the donut by construction (both sides ultimately come from the same ES data).

6.7 `_monthKey` — and the timezone bug it fixes

```
_monthKey: function(key) {
  var str = String(key);
  if (/^\d{10,}$/.test(str)) {
    var d = new Date(parseInt(str, 10));
    if (!isNaN(d.getTime())) {
      return d.getUTCFullYear() + '-' + ('0' + (d.getUTCMonth() +
1)).slice(-2);
    }
  }
  return str.split('T')[0].substring(0, 7);
},
```

- `/^\d{10,}$/` — "the whole string is 10+ digits" = an epoch-milliseconds key (ES date histograms commonly return these).
- **The UTC fix (a real latent bug found by testing):** ES buckets sit at UTC midnight. `new Date(epochMs).getMonth()` converts to the BROWSER's local time first — in any timezone behind UTC (all of the Americas), "July 1, 00:00 UTC" becomes "June 30 evening" locally, and the month key comes out as JUNE. Every histogram value would then match the wrong month. Using `getUTCFullYear()` / `getUTCMonth()` reads the UTC parts directly — correct in every timezone. (India, UTC+5:30, is ahead of UTC and happened to be safe — which is exactly why this bug hid during dev and was only caught by explicitly testing under `TZ=America/New_York`.) The same UTC treatment was applied to `_toMonthYear`'s epoch branch so key and label can never disagree.
- Non-numeric keys: `'2026-07-14T...'` → split at `'T'` → first 7 chars → `'2026-07'`. One helper, every key format, one canonical output.

6.8 Worked example — full trace, three months

Period applied: `startDate='2026-07-01', endDate='2026-09-30'`.

`_buildMonthList:`

```
[ {key:'2026-07', label:'Jul-26', start:'2026-07-01 00:00:00',
  end:'2026-07-31 23:59:59', migrated:0},
  {key:'2026-08', label:'Aug-26', start:'2026-08-01 00:00:00',
  end:'2026-08-31 23:59:59', migrated:0},
  {key:'2026-09', label:'Sep-26', start:'2026-09-01 00:00:00',
  end:'2026-09-30 23:59:59', migrated:0} ]
```

(July's `lastOfMonth = new Date(2026, 7, 0) = July 31` — day-zero trick.)

`_fetchMonthlyMigrated:` `_monthGen 1→2, gen=2; immediate render:`

```
monthlyMigrated = [ {Jul-26, migrated:0, nonMigrated:8000,
  all:8000}, ... ]
```

(with docsPerMonth = [8000, 9000, 7000] already in — bars appear all-red first, honest "not counted yet" state). Three fetches fire:

```
.../fid_MIGRATED/execute?dublincore_created=2026-07-01%2000%3A00%3A00&dublincore_created_1=2026-07-31%2023%3A59%3A59&pageSize=1
... (Aug, Sep same shape)
```

Responses land (any order): July {resultsCount: 2000} → gen matches → m.migrated = 2000 → _renderMonthly:

Jul-26: migrated 2000, nonMigrated 8000-2000 = 6000

Aug 3000 → nonMig 6000. Sep 1500 → nonMig 5500. Final:

Month	Migrated (green)	Non-migrated (red)	Total
Jul-26	2,000	6,000	8,000
Aug-26	3,000	6,000	9,000
Sep-26	1,500	5,500	7,000

Sums: migrated 6,500 / non-migrated 17,500 / total 24,000 — **exactly consistent with what the donut computes** from fid_MIGRATED / fid_TOTAL over the same range. The two visualisations can no longer disagree, which was the original bug report that started this entire chain.

PART 7 — SLOTS 4 & 5: TOP 5 USE CASES (docs / storage, with migration %)

7.1 Markup, line by line (Slot 4; Slot 5 is identical with storage)

```
<nuxeo-card class="card-half" heading="Top 5 Use Cases">
  <div class="usecase-subhead">by document volume</div>
  <template is="dom-if" if="[[!_isEmpty(topByDocs)]]">
    <div class="usecase-list">
      <div class="usecase-head-row">
        <span class="uc-rank-h">#</span>
        <span class="uc-name-h">USE CASE</span>
        <span class="uc-val-h">DOCS</span>
        <span class="uc-status-h">MIGRATED %</span>
      </div>
      <template is="dom-repeat" items="[[topByDocs]]" as="row">
        <div class="usecase-row">
          <span class="uc-rank">[[_inc(index)]]</span>
          <div class="uc-main">
            <span class="uc-name">[[row.label]]</span>
            <div class="uc-bar">
              <div class="uc-bar-fill" style$="
[[_barStyle(row.value, topByDocsMax)]]"></div>
            </div>
            <span class="uc-val">[[_formatCount(row.value)]]</span>
            <span class="uc-status">[[_statusPct(row.type,
_docCountByType, _statusVersion)]]</span>
          </div>
        </div>
      </template>
    </div>
  </template>
  <template is="dom-if" if="[[!_isEmpty(topByDocs)]]">
    <div class="chart-empty">No documents found in this period</div>
  </template>
</nuxeo-card>
```

```

</template>
</nuxeo-card>

```

- **dom-repeat** — stamps one row per array entry; as="row" names the loop variable; index is the automatic 0-based position.
- `[[_inc(index)]]` — return index + 1; — ranks display 1-5, not 0-4.
- **style\$= + _barStyle** — the proportional progress bar:

```

_barStyle: function(value, max) {
    var pct = (max && max > 0) ? Math.round((value / max) * 100) :
0;
    if (pct < 4 && value > 0) pct = 4; // keep a sliver visible
    return 'width:' + pct + '%';
},

```

Width as a % of the LIST'S LARGEST value (topByDocsMax), so rank 1 is always full-width and the rest are proportional. The `pct < 4` clamp keeps a tiny-but-nonzero use case visibly present rather than invisibly 0-wide. Remember: `style$=` (attribute binding) is mandatory here.

- Slot 5 differs only in: subhead "by storage added", items="[[topByStorage]]", `_formatStorage(row.value)` for the value column, `topByStorageMax` for the bar.

7.2 The data pipeline — from aggregation to ranked rows

Two ES aggregations feed these slots: `byDocType` (doc counts per type) and `storageByDocType` (byte sums per type), each `group-limit="10"`. Two observers process them:

```

_computeTopByDocs: function(data) {
    var map = {};
    (data || []).forEach(function(entry) {
        if (entry && entry.key) map[entry.key] = entry.value || 0;
    });
    this._docCountByType = map;
    var top = this._topN(data, 5);
    this.topByDocs = top;
    this.topByDocsMax = top.length ? top[0].value : 0;
    this._fetchTypeMigrated();
},

_computeTopByStorage: function(data) {
    var top = this._topN(data, 5);
    this.topByStorage = top;
    this.topByStorageMax = top.length ? top[0].value : 0;
    this._fetchTypeMigrated();
},

```

`_computeTopByDocs` does three jobs: (1) builds `_docCountByType` — the {type → total docs} lookup that the migration % denominator uses; (2) derives the top-5 list; (3) triggers the per-type migrated fetches. The storage twin does (2) and (3).

```

_topN: function(data, n) {
    if (!data || !data.length) return [];
    var self = this;
    var copy = data.slice().sort(function(a, b) {
        return (b.value || 0) - (a.value || 0);
    });
    return copy.slice(0, n).map(function(entry) {
        return {
            type: entry.key,
            label: self._friendlyDocType(entry.key),
            value: entry.value || 0
        };
    });
}

```

```
});
},
```

- **data.slice() before .sort()** — .sort() mutates in place; slicing first copies the array so the original aggregation property is never reordered behind Polymer's back. A subtle correctness habit worth naming.
- The comparator (b.value - a.value) sorts DESCENDING; .slice(0, n) takes the top 5; the map reshapes buckets into row objects, keeping the raw type (needed as the lookup key for percentages) alongside the display label.

```
_friendlyDocType: function(type) {
  if (!type) return 'Unknown';
  var map = { 'File': 'File', 'Note': 'Note', ...,
    'BrokerageServiceDoc': 'Brokerage Service Doc' };
  if (map[type]) return map[type];
  return String(type).replace(/[a-z]([A-Z])/g, '$1 $2');
},
```

Known types mapped explicitly; anything else gets the camelCase-splitter regex — "insert a space between a lowercase letter and the uppercase letter that follows it": InvoiceArchiveDoc → Invoice Archive Doc. New document types added in production get readable labels automatically, no code change.

7.3 The migration % — the fetch pipeline, line by line

```
_typeMigrated: { type: Object, value: function() { return {}; } },
_statusVersion: { type: Number, value: 0 },
_typeGen:       { type: Number, value: 0 },

_fetchTypeMigrated: function() {
  var TYPE_PARAM = 'system_primaryType'; // Studio-generated param
  name for the ecm:primaryType predicate

  var seen = {};
  var types = [];
  (this.topByDocs || []).concat(this.topByStorage ||
[]).forEach(function(r) {
    if (r && r.type && !seen[r.type]) { seen[r.type] = true;
types.push(r.type); }
  });

  var sig = types.join('|') + '::' + (this.startDate || '') + '::' +
(this.endDate || '');
  if (sig === this._typeFetchSig) { return; }
  this._typeFetchSig = sig;

  if (!types.length || !this.startDate || !this.endDate) {
    this._typeMigrated = {};
    this._statusVersion++;
    return;
  }

  this._typeGen = (this._typeGen || 0) + 1;
  var gen = this._typeGen;
  var startTs = this._toTimestamp(this.startDate, false);
  var endTs = this._toTimestamp(this.endDate, true);
  this._typeMigrated = {};
  this._statusVersion++;
  var self = this;

  types.forEach(function(t) {
    var url = '/nuxeo/api/v1/search/pp/fid_MIGRATED_BY_TYPE/execute'
      + '?dublincore_created=' + encodeURIComponent(startTs)
```



```

        + '&dublincore_created_1=' + encodeURIComponent(endTs)
        + '&' + TYPE_PARAM + '=' + encodeURIComponent(t)
        + '&pageSize=1';
    fetch(url, { headers: { 'Accept': 'application/json' },
credentials: 'include' })
        .then(function(resp) {
            if (!resp.ok) { throw new Error('fid_MIGRATED_BY_TYPE HTTP '
+ resp.status); }
            return resp.json();
        }).then(function(json) {
            if (self._typeGen !== gen) { return; }
            var count = json && typeof json.resultsCount === 'number' ?
json.resultsCount : 0;
            var map = {};
            var src = self._typeMigrated || {};
            for (var k in src) { if (src.hasOwnProperty(k)) { map[k] =
src[k]; } }

            map[t] = count > 0 ? count : 0;
            self._typeMigrated = map;
            self._statusVersion++;
        }).catch(function(err) {
            if (window.console) { console.error('Per-type migrated fetch
failed for ' + t, err); }
        });
    });
},
},

```

Reading it top to bottom:

- **TYPE_PARAM** — the one adjustable constant: the Studio-generated name for the type predicate. Because ecm: is Nuxeo's SYSTEM schema, Studio named it `system_primaryType` (not the guessed `ecm_primaryType`) — discovered during setup and set here.
- **Distinct-type collection with dedup** — both top-5 lists concatenated; the seen object drops repeats (a type in both lists — common — is fetched once, not twice). Worst case ≤ 10 types.
- **The signature (sig) dedup** — the exact type set + date range is fingerprinted. Both `_computeTopByDocs` AND `_computeTopByStorage` call this method (two aggregations, two observer fires per load) — without the signature, every load would fetch everything twice. Same types + same range → second call returns immediately. Any change in either → refetch.
- **Empty guard** — no types or no dates: clear the map, bump the version (so 0% renders), stop.
- **gen race guard** — identical pattern to the monthly fetches: stale responses from an outdated range/type-set discard themselves.
- **The copy-then-assign map update** — the response handler copies the existing map, adds this type's count, and assigns the NEW object to this `_typeMigrated`. Mutating the existing object in place would not produce a change Polymer can act on; a fresh reference always does.
- **_statusVersion++** — the render trigger for the percentage cells (explained next).

7.4 _statusPct — and the _statusVersion trick

```

<span class="uc-status">[[_statusPct(row.type, _docCountByType,
_statusVersion)]]</span>

_statusPct: function(type, docCount, _v) {
    var total = (docCount || {})[type] || 0;
    if (total <= 0) return '0%';
    var migrated = (this._typeMigrated || {})[type] || 0;
    var pct = Math.round((migrated / total) * 100);
    return pct + '%';
},

```

- Numerator: `this._typeMigrated[type]` (read internally).
Denominator: `_docCountByType[type]` (passed in). `total <= 0` guard → '0%', never NaN.
- **The `_v` argument is never used inside the function — so why pass it?** This is a deliberate Polymer idiom worth explaining to your lead: a computed binding only re-runs when one of its LISTED arguments changes. `row.type` and `_docCountByType` don't change when a migrated count lands — so without a third trigger, the cells would render once at 0% and never update. `_statusVersion` is a counter bumped after every landed fetch; listing it as an argument forces Polymer to re-call `_statusPct` for every row, which then reads the fresh `_typeMigrated` map. A version counter as a "recompute now" signal.

7.5 Worked example — slots 4 & 5 end to end

byDocType lands:

```
[ {key:'File',value:6000}, {key:'Invoice',value:4000},
{key:'Note',value:2500},
  {key:'Picture',value:1500}, {key:'Contract',value:900},
{key:'Video',value:516} ]
```

_computeTopByDocs:

```
_docCountByType = {File:6000, Invoice:4000, Note:2500, Picture:1500,
Contract:900, Video:516}
_topN(...,5) → sorted desc, top 5, reshaped:
[ {type:'File', label:'File', value:6000},
  {type:'Invoice', label:'Invoice', value:4000},
  {type:'Note', label:'Note', value:2500},
  {type:'Picture', label:'Picture', value:1500},
  {type:'Contract',label:'Contract',value:900} ]
topByDocsMax = 6000
```

Rows render: rank 1 File bar width:100%, Invoice `_barStyle(4000,6000)` → width:67%, Contract → width:15%; values 6K / 4K / 2.5K / 1.5K / 900. Percent cells initially 0%.

_fetchTypeMigrated: suppose `topByStorage` adds one more distinct type (Video) → 6 fetches:

```
.../fid_MIGRATED_BY_TYPE/execute?dublincore_created=2026-07-01
00:00:00
&dublincore_created_1=2026-09-30
23:59:59&system_primaryType=File&pageSize=1
```

Responses land: File→2700, Invoice→1000, Note→625, Picture→150, Contract→900, Video→0. Each: map copied + extended, `_statusVersion` 1→2→...→7.

Every bump re-runs every `_statusPct` cell:

```
File:      2700/6000 → 45%      Invoice: 1000/4000 → 25%
Note:      625/2500 → 25%      Picture:  150/1500 → 10%
Contract:  900/900  → 100%
```

Both slots' MIGRATED % columns fill in progressively as responses arrive — each percentage a pure ES-provider count over an ES-aggregation total.

7.6 CSS

```
.usecase-row { display: flex; align-items: center; gap: 10px;
padding: 9px 0;
```

```

        border-bottom: 1px solid #f0f3f1; }
.uc-rank { width: 20px; color: #9aa39c; font-weight: 600; }
.uc-main { flex: 1; min-width: 0; }
.uc-name { font-size: 13px; color: #20302a; white-space:
nowrap;
        overflow: hidden; text-overflow: ellipsis; }
.uc-bar { height: 6px; background: #eef2ef; border-radius:
3px; margin-top: 4px; }
.uc-bar-fill { height: 100%; background: linear-
gradient(90deg,#43a047,#2e7d32);
        border-radius: 3px; }
.uc-val { width: 64px; text-align: right; font-weight: 600; }
.uc-status { width: 84px; text-align: right; color: #2e7d32;
font-weight: 600; }

```

Notables: `min-width: 0` on the flexing middle column — without it, a long type name refuses to shrink and blows the row layout; with it, the ellipsis truncation works. The bar fill is a subtle green gradient; fixed widths on the two right columns keep every row's numbers vertically aligned.

PART 8 — SLOT 6: GROWTH VS BASELINE (leadership's chart)

(This slot has its own dedicated 27-page deep-dive — [BASELINE_AND_EMAIL_EXPLAINED.pdf](#). This section is the complete but condensed line-by-line; use the dedicated PDF for the extended worked traces.)

8.1 The independent data feeds

```

<nuxeo-repository-data start-date="[[baselineStart]]" end-date="
[[baselineEnd]]"
  with-date-intervals="month" date-field="dc:created"
  data="{baselineDocsPerMonth}" index="[[index]]"></nuxeo-
repository-data>

<nuxeo-repository-data start-date="[[baselineStart]]" end-date="
[[baselineEnd]]"
  with-date-intervals="month" date-field="dc:created"
  metrics="sum(file:content.length)"
  data="{baselineStoragePerMonth}" index="[[index]]"></nuxeo-
repository-data>

```

The defining design decision: these bind to `baselineStart/baselineEnd` — **NOT the page's `startDate/endDate`**. Leadership's fixed reference range is completely decoupled from whatever period someone is browsing; changing the page period never touches this chart, and vice versa.

8.2 The picker and Apply, line by line

```

<div class="baseline-picker">
  <div class="date-field"><label>Baseline month</label>
    <input type="month" value="{baselinePick:change}" /></div>
  <div class="date-field"><label>Up to</label>
    <input type="month" value="{baselineEndPick:change}" /></div>
  <div class="date-field"><label>&nbsp;</label>
    <button type="button" class="custom-submit" on-
tap="_applyBaseline">Apply</button></div>
  <span class="baseline-picker-status">[[baselinePickStatus]]</span>
</div>

```

Native `<input type="month">` pickers (browser-drawn calendar). Two property pairs exist deliberately: `baselinePick/baselineEndPick` change on every picker interaction; the `QUERIES` bind to `baselineStart/baselineEnd`, which change **ONLY** inside `_applyBaseline` — so browsing the picker costs zero Elasticsearch queries.

```

_applyBaseline: function() {
  var pick    = (this.baselinePick || '').trim();      // 'YYYY-MM'
  var endPick = (this.baselineEndPick || '').trim();
  var MONTH = /^\\d{4}-\\d{2}$/;
  if (!MONTH.test(pick)) { this.baselinePickStatus = 'Pick a
baseline month first.'; return; }
  if (!MONTH.test(endPick)) { this.baselinePickStatus = 'Pick an end
month first.'; return; }
  if (endPick < pick) {
    this.baselinePickStatus = 'End month cannot be before the
baseline month.'; return;
  }
  this.setProperties({
    baselineStart: pick + '-01',
    baselineEnd:   this._monthEndDate(endPick),
    baselineLabel: this._monthYearLabel(pick),
    baselineEndLabel: this._monthYearLabel(endPick)
  });
  this.baselinePickStatus = '';
},

```

- The regex `^\\d{4}-\\d{2}$` validates the shape (4 digits, dash, 2 digits).
- **`endPick < pick` — comparing strings with `<` is safe here** precisely because `'YYYY-MM'` is fixed-width and zero-padded: character-by-character string order equals chronological order.
- Any month, past or future, is allowed — a deliberate leadership choice; an empty future range renders an empty chart, not an error (every downstream helper has guards).
- The single `setProperties` commits both range ends together → both baseline queries fire exactly **ONCE** per Apply, never with a half-updated range.

```

_monthEndDate: function(pick) {
  if (!/^\\d{4}-\\d{2}$/.test(pick || '')) return '';
  var parts = pick.split('-');
  var last = new Date(parseInt(parts[0], 10), parseInt(parts[1],
10), 0);
  return this._toISODate(last);
},

```

The day-zero trick again: `new Date(2026, 2, 0)` = Feb 28/29 2026 — leap-year-safe last-day-of-month in one line.

`ready()` (the Polymer lifecycle hook that runs once at attach) defaults the "Up to" end to the current month via the same `batched-setProperties` pattern, so the first render shows baseline → today.

8.3 The chart and the growth table

```

<chart-bar
  labels="[[_monthLabels(baselineStoragePerMonth)]]"
  values="[[_baselineValues(baselineStoragePerMonth,
baselineDocsPerMonth)]]"
  series="[[_baselineSeries(baselineStoragePerMonth)]]"
  options="[[_overallOptions(baselineStoragePerMonth)]]">
</chart-bar>

```

Reuses `_monthLabels` and `_overallOptions` from the Growth chart — same visual language on purpose. `_baselineValues` is the by-key alignment recipe (docs looked up by month key against the storage

order, unit auto-scaled); _baselineSeries labels the legend with the SAME _storageUnit so unit and numbers can't disagree.

```
<template is="dom-if" if="[_hasBaseline(_baselineGrowth)]">
  <table class="baseline-table"> ...
    <template is="dom-repeat" items="[_baselineGrowth]">
      <tr>
        <td>[[item.month]]</td>
        <td class="num-cell">[[item.docs]]</td>
        <td class$="[_deltaClass(item.docsPct)]">[[item.docsPct]]
      </td>
        <td class="num-cell">[[item.storage]]</td>
        <td class$="[_deltaClass(item.storagePct)]">
[[item.storagePct]]</td>
      </tr>
    </template>
```

The table is driven by ONE computed property:

```
_baselineGrowth: {
  type: Array, value: function() { return []; },
  computed: '_computeBaselineGrowth(baselineStoragePerMonth,
baselineDocsPerMonth)'
```

Inside _computeBaselineGrowth, the essential lines:

```
var baseDocs = docMap[storage[0].key] || 0; // the FIRST bucket
IS the baseline
var baseStorage = storage[0].value || 0;

function pct(current, base, fmt) {
  if (!base) {
    return current > 0 ? '+' + fmt(current) + ' new' : '0%';
  }
  var change = ((current - base) / base) * 100;
  var rounded = Math.round(change * 10) / 10;
  return (rounded > 0 ? '+' : '-') + rounded + '%';
}
```

- The baseline month is never hardcoded — because the query STARTS at the picked month, storage[0] is the baseline by construction; repick and everything follows.
- **The zero-baseline fix:** (current-base)/base is undefined at base=0 (division by zero). Instead of n/a, the branch shows the ABSOLUTE increase — +116 new documents, +452.9 KB new storage — computable, honest, and (because it starts with +) still colored green by _deltaClass. This fixed a real reported case (a January-2026 baseline with zero documents).
- Row 0 is special-cased (i === 0 → 'baseline') — comparing the baseline to itself would always be a meaningless +0%.

```
_deltaClass: function(pctText) {
  if (!pctText || pctText === 'baseline' || pctText === 'n/a')
return 'delta-cell delta-flat';
  if (pctText.charAt(0) === '+') return 'delta-cell delta-up';
  if (pctText.charAt(0) === '-') return 'delta-cell delta-down';
  return 'delta-cell delta-flat';
},
```

Color chosen purely from the first character; the stylesheet maps delta-up → green, delta-down → red, delta-flat → grey.

Table CSS notables: th/td:nth-child(3) { border-right: 2px solid #20302a } draws the vertical rule separating the documents block from the storage block (3rd column = end of the docs pair); .num-cell { font-variant-numeric: tabular-nums } forces equal-width digits so number columns align like a financial statement.

8.4 Micro worked example

Baseline July 2026 → Sep 2026, docs [1000, 1500, 750], storage [500 MB, 750 MB, 400 MB]:

```
baseDocs=1000, baseStorage=524288000
Aug: pct(1500,1000)= +50%      pct(750MB,500MB)= +50%
Sep: pct(750,1000) = -25%      pct(400MB,500MB)= -20%
```

Table: Jul *baseline/baseline* · Aug **+50%** / **+50%** (green) · Sep **-25%** / **-20%** (red). Full multi-page traces, including the zero-baseline case, live in the dedicated baseline PDF.

PART 9 — THE EMAIL REPORT (condensed; full version in its own PDF)

One button (on-tap="_emailReport"), one status property (emailStatus), one large method. The six stages:

1. **Numbers** — reads totalCount, migratedCount, computes non-migrated and %; .bind(this) locks _withCommas/_formatStorage so they can be called as bare functions in callbacks.
 2. **HTML body** — string-concatenated tables with INLINE styles (email clients strip <style> blocks — inline is the only reliable styling in email). Reuses the SAME computed data as the screen — including _baselineGrowth and _statusPct — one source of truth, three destinations (screen, email, PDF).
 3. **sendMail(tempDoc)** — a closure over the built HTML; POSTs Nuxeo's built-in Notification.SendMail automation op (input: 'doc:/ — root as context; with a PDF: input: 'doc:<uid>' + files: 'file:content'). Why a built-in op from the browser: the server's Nashorn scripting is broken on this instance, so custom server-side scripts are off the table; built-in Java operations work fine.
 4. **Chart capture** — activeView = 'overall', load jsPDF on demand (_ensurePdfLibs: cached window.jspdf check, else inject a <script> tag), wait 500 ms for the tab's chart to redraw, then shot(heading): find the card by heading, take its <canvas>, draw it onto a temp canvas over a white fill (kills PNG transparency that renders black in PDFs), export via `canvas.toDataURL('image/png')`. **Why native toDataURL and not html2canvas — a production war story:** html2canvas froze the browser's main thread on these Chart.js canvases for 30+ minutes ("Capturing... 2/3" forever). toDataURL copies pixels off an existing canvas — O(pixels), no DOM walking, cannot hang. **Why PNG not JPEG — measured, not assumed:** for flat-color charts JPEG compressed ~5× SMALLER than JPEG (JPEG is built for photos). A JPEG "optimisation" was tried, measured, reverted.
 5. **PDF assembly** — jsPDF, cover page + one page per captured chart; numbers that live OUTSIDE the canvas (the donut's center %, the summary stats) are re-drawn as PDF TEXT at computed coordinates — closing the gap toDataURL leaves. pdf.output('blob') → size guard: > 4000 KB → throw (a real 5.4 MB attachment was once silently refused by the mail server).
 6. **Attach + send** — Nuxeo's three-step blob dance: POST /upload/ (batch id) → POST /upload/<id>/0 (the bytes) → Document.Create with file:content referencing the batch → sendMail(tempDoc). **One .catch() closes the whole chain:** ANY failure — library load, capture, oversize, any upload step — falls through to sendMail(null): the numbers-and-tables email still goes out. The report can degrade but never silently vanish.
-

PART 10 — THE DEBUGGING DISCOVERIES (the war-story appendix)

These are the findings your lead is most likely to probe — each one changed the architecture.

10.1 results-count vs number-of-results (the original zero bug)

The Polymer element exposes the count as **results-count**; an early version bound `number-of-results` — a property that doesn't exist — so every count silently stayed 0. Lesson: element attribute names must match the element's actual API; silence, not errors, is the failure mode.

10.2 pageSize=1 vs pageSize=0 (the counting economics)

Two SEPARATE contracts in Nuxeo's PageProvider interface: `setPageSize` governs how many documents are FETCHED and hydrated (0 = "no pagination, return all," capped at 1000 — an EXPORT tool); `getResultsCount` is the total, computed independently. On an ES-backed provider the total is always available regardless of page size (documented example: `pageSize:5` returning `resultsCount:99891`, ES 22 ms vs DB 324 ms for the same query). So `pageSize=1` returns the identical exact count while hydrating ONE document instead of up to 1000. "Hydration" = building a full DocumentModel (fields, schemas, ACLs) and serializing it — the cost a count never needs. Any "pageSize=0 is best" advice conflates it with `SearchService.limit(0)` — a different API where 0 means "count only, load NOTHING," the opposite meaning. Every count surface in this dashboard uses `page-size/pageSize = 1`.

10.3 Why named Studio providers (and not ad-hoc NXQL)

Ad-hoc query= NXQL routes through `REST_API_SEARCH_ADAPTER`, which is ES-backed only if `elasticsearch.override.pageproviders` in `nuxeo.conf` includes it — a shared server setting that can drift, get reverted, or simply be missing, silently sending every count to the database (slower AND potentially inexact: `CoreQueryDocumentPageProvider` is documented as possibly unable to compute an exact total, returning a negative `UNKNOWN_SIZE_AFTER_QUERY`). A Studio provider with "Use Elasticsearch index" checked carries its ES routing in its OWN definition, versioned and deployed with the project. The final dashboard needs NOTHING from `nuxeo.conf`.

10.4 The 32k bug — why the monthly chart uses fetch()

Three attempts to run per-month `fid_MIGRATED` queries as `<nuxeo-page-provider>` elements inside a `dom-repeat` failed: (a) all providers sharing one `results-count` target meant Polymer's dirty-checking DEDUPED equal values — two months with the same count → the second's change event never fired, count stuck at 0; (b) writing into array elements via the event model didn't reliably trigger the computed chart; (c) worst: the per-item named-query-params binding failed to

deliver parameters to the stamped providers — and **with named-parameter providers, missing parameters are not an error: Nuxeo silently DROPS those predicates**. Every month's query ran with NO date filter and returned the all-time migrated count — 32,000 — which is exactly what the broken chart displayed. Diagnosis came from reading the number itself: 32k > any possible period total ⇒ the date predicates weren't applying. **The fix:** abandon the element-in-repeat pattern entirely; issue the requests as explicit `fetch()` URLs — the byte-for-byte Bruno-proven request — where a parameter physically cannot be dropped, every call is visible in the Network tab, and a generation counter handles races.

10.5 The inflation bug (the first monthly bug)

The original monthly chart derived per-month totals by matching the histogram to months by key, then SCALING the matched values so they summed to `totalCount`. If even one bucket key misaligned, the scaling INFLATED the surviving months — a month showing 32k against a 10k period total. Fix: per-month totals now use the histogram values DIRECTLY (each month's non-migrated = its own total – its own migrated); no cross-month scaling exists in the monthly path, so inflation is structurally impossible. The reconcile technique survives only on the Overall chart (10.6).

10.6 Where reconcile-scaling still lives, and why that's OK

`_reconcileDocs` still adjusts the Overall chart's document line so the months sum exactly to the KPI's `totalCount` — there it's cosmetic consistency with an authoritative number, not an input to a per-month subtraction, so the failure mode of 10.5 doesn't apply. Documented here so nobody mistakes it for an oversight.

10.7 The timezone bug

ES buckets sit at UTC midnight; converting epoch keys with local-time `getMonth()` shifts the month BACKWARD in any timezone behind UTC (all the Americas) — histogram values would match the wrong months. Hidden during development because India (UTC+5:30) is ahead of UTC; caught only by explicitly testing under `TZ=America/New_York`. Fix: `getUTCFullYear()` / `getUTCMonth()` in both `_monthKey` and `_toMonthYear`.

10.8 where= on nuxeo-repository-data — the aggregation that wasn't

A more elegant design existed: ONE filtered aggregation (`where="fid_migration:...IS NOT NULL" + grouped-by / date-intervals`) replacing all the per-month and per-type count calls. Nuxeo release notes documented the `where=` attribute — but on this LTS 2023 instance it returned empty (slots 4/5 all 0%). Both code paths were built and triple-verified; live testing chose the fallback. Lesson: documentation describes A version; only YOUR instance describes your version — test before trusting, in Bruno, with known numbers.

10.9 The verification methodology (how every change shipped)

Every change ran a triple check before delivery: **R1 static** — extract the `<script>` → `node --check`; scan for missing helpers/handlers/observers, duplicates, nested bindings, tag balance, `page-size=1` everywhere, the migration predicate intact. **R2 behavioral** — extract the REAL method bodies verbatim from the file and execute them in Node against realistic data — including mocked fetch for the REST pipelines, duplicate-value scenarios, race scenarios, and multiple timezones. **R3 regression + scale** — every downstream consumer still wired, query budget counted, bounded-ness confirmed. Recurring false-positive: test regexes counting a mention in a COMMENT as code (e.g. a doc comment naming `<nuxeo-page-provider>`) — every flag was investigated, most were counting artifacts, and the discipline of checking rather than waving off is itself the lesson.

10.10 Production scale posture (the billion-document answer)

Worst case **31 requests per load**: 7 aggregations (cost scales with buckets — ≤ 12 months / ≤ 10 groups — not corpus size) + 2 provider counts + ≤ 12 monthly fetches + ≤ 10 per-type fetches — every one count-only or bucket-only; **zero document hydration anywhere**. Race guards and signature dedup prevent storms; batched `setProperties` prevents double-fires; all count surfaces are Studio-ES-guaranteed. The one thing to eyeball on production day one: that counts far above 10,000 come back exact (ES's `index.max_result_window` territory) — verified empirically to 32k on the dev instance; if production ever shows capped/negative large counts, the fix is the server-side `org.nuxeo.elasticsearch.provider.maxResultWindow` setting, not dashboard code.

PART 11 — GLOSSARY (quick reference)

Term	Meaning
<code>[[x]] / {{x}}</code>	one-way display binding / two-way writable binding
<code>[[_fn(a,b)]]</code>	computed binding — re-runs when a listed arg changes
computed property	JS-declared auto-derived property
observer	method run when listed properties change (side effects)
<code>dom-if / dom-repeat</code>	conditional / repeated template stamping
<code>class\$= / style\$=</code>	attribute (not property) binding — required for native attrs
<code>value: function() {return [];}</code>	mandatory function-wrapped array/object default
<code>setProperties({...})</code>	batched multi-property set → one notification cycle
<code>on-tap</code>	Polymer tap/click event binding
<code>::change</code>	write-back on that DOM event (input bindings)
<code>ready()</code>	lifecycle hook, runs once when the element is ready
aggregation	ES-side bucketed computation; returns <code>{key,value}</code> buckets
date histogram	aggregation bucketed per time interval

terms aggregation	aggregation bucketed per distinct field value
page provider	Nuxeo's named, paginated query definition
named parameters	Studio-provider params passed as name=value
resultsCount	the provider's total match count (independent of pageSize)
hydration	building + serializing a full DocumentModel — the cost pageSize controls
ecm:	Enterprise Content Management — Nuxeo's built-in system schema
Promise / .then / .catch	async value / next step / one error handler for the chain
closure	inner function remembering its outer scope's variables
var self = this	captures the component for use inside callbacks
generation counter	int captured per request wave; stale responses self-discard
signature dedup	fingerprint of inputs; identical inputs → skip refetch
guard clause	leading if (!data) return ... — survives pre-data renders
day-zero trick	new Date(y, m+1, 0) = last day of month m
tabular-nums	CSS: equal-width digits so number columns align

*End of the book. Companion deep-dives:
 BASELINE_AND_EMAIL_EXPLAINED.pdf (slot 6 + email, extended traces),
 PERFORMANCE_COMPARISON.pdf (before/after optimization), and the three
 provider HOW-TO guides.*